

# Fundamentos de Frontend Moderno

HTML • CSS • JavaScript • React

Resumo consolidado dos principais conceitos de desenvolvimento web moderno,  
elaborado para consulta rápida da equipe.

---

Material de apoio ao estudo do projeto

# Sumário

- 1. Visão geral do estudo
- 2. HTML5 Semântico
- 3. CSS Moderno (Flexbox, Grid, variáveis e responsividade)
- 4. JavaScript ES6+ (sintaxe moderna e assincronismo)
- 5. Manipulação do DOM
- 6. React (Componentes, Hooks e Estado)
- 7. Boas práticas: Acessibilidade (a11y) e Performance
- 8. Checklist final e próximos passos

# 1. Visão Geral do Estudo

Este material resume os fundamentos e boas práticas do desenvolvimento Frontend moderno, cobrindo a estrutura (HTML), a estilização (CSS) e o dinamismo (JavaScript) da web, além da integração com o framework React para construção de interfaces eficientes, acessíveis e responsivas. O objetivo é servir como referência rápida para toda a equipe durante o desenvolvimento do projeto.

## Objetivos do estudo

- Dominar a estrutura semântica do HTML5 e a estilização avançada com CSS3, Flexbox e Grid;
- Entender a manipulação do DOM e o assincronismo em JavaScript (Promises, Async/Await);
- Compreender os conceitos fundamentais do React (Componentes, Hooks e Estado);
- Aplicar boas práticas de responsividade, acessibilidade (a11y) e performance web.

## Resultado esperado

Que a equipe seja capaz de planejar, estruturar e implementar componentes de interface funcionais e responsivos, consumindo APIs REST e garantindo boas práticas de código limpo e usabilidade no projeto final.

## 2. HTML5 Semântico

HTML semântico significa usar tags que descrevem o **significado** do conteúdo, e não apenas sua aparência. Isso melhora o SEO (mecanismos de busca entendem melhor a página), a acessibilidade (leitores de tela conseguem navegar com mais precisão) e a manutenção do código.

### Principais tags semânticas

| Tag                   | Uso                                                           |
|-----------------------|---------------------------------------------------------------|
| <header>              | Cabeçalho da página ou de uma seção (logo, título, navegação) |
| <nav>                 | Bloco de links de navegação principal                         |
| <main>                | Conteúdo principal e único da página (só um por página)       |
| <section>             | Agrupar conteúdo relacionado sob um mesmo tema                |
| <article>             | Conteúdo independente e reutilizável (post, notícia, card)    |
| <aside>               | Conteúdo complementar (barra lateral, anúncio, dica)          |
| <footer>              | Rodapé da página ou de uma seção                              |
| <figure>/<figcaption> | Imagem ou mídia com legenda associada                         |

### Exemplo de estrutura semântica

```
<body>
  <header>
    <h1>Meu Site</h1>
    <nav><a href="#sobre">Sobre</a></nav>
  </header>

  <main>
    <section id="sobre">
      <h2>Sobre nós</h2>
      <p>Texto explicativo...</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 - Todos os direitos reservados</p>
  </footer>
</body>
```

### Boas práticas de HTML semântico

- Use apenas **uma tag <h1>** por página, representando o título principal;
- Prefira **<button>** para ações e **<a>** apenas para navegação (links);
- Sempre inclua o atributo **alt** em imagens, descrevendo seu conteúdo;
- Utilize **<label>** associado a todo campo de formulário (via atributo *for*);
- Evite abusar de **<div>** e **<span>** quando existe uma tag semântica equivalente.

## 3. CSS Moderno

O CSS moderno vai muito além de cores e fontes: envolve sistemas de layout (Flexbox e Grid), variáveis reutilizáveis e técnicas de responsividade para que a interface funcione bem em qualquer tamanho de tela.

### Flexbox — layout em uma dimensão (linha ou coluna)

Ideal para alinhar itens em uma única direção (linha ou coluna), como menus, cards em fileira ou centralização de elementos.

```
.container {
  display: flex;
  justify-content: space-between; /* alinhamento horizontal */
  align-items: center;           /* alinhamento vertical */
  gap: 16px;                    /* espaço entre os itens */
  flex-wrap: wrap;              /* quebra linha se não couber */
}
```

- **justify-content:** alinha os itens no eixo principal (ex: flex-start, center, space-between);
- **align-items:** alinha os itens no eixo cruzado (ex: center, stretch, flex-end);
- **flex: 1** em um item faz com que ele cresça para ocupar o espaço disponível.

### CSS Grid — layout em duas dimensões (linhas e colunas)

Ideal para estruturas mais complexas, como o layout geral de uma página (header, sidebar, conteúdo, footer).

```
.grid {
  display: grid;
  grid-template-columns: 1fr 3fr; /* 2 colunas: sidebar + conteúdo */
  grid-template-rows: auto 1fr auto;
  gap: 20px;
}
```

■ **Dica prática:** Regra prática: use **Flexbox** para alinhar componentes internos (ex.: itens de um card) e **Grid** para montar a estrutura geral da página.

### Variáveis CSS (Custom Properties)

Permitem centralizar valores reutilizáveis (cores, espaçamentos, fontes), facilitando manutenção e temas.

```
:root {
  --cor-primaria: #2D5BE3;
  --espacamento-padrao: 16px;
}

.botoao {
  background-color: var(--cor-primaria);
  padding: var(--espacamento-padrao);
}
```

### Media Queries e Mobile First

Media queries aplicam estilos diferentes conforme o tamanho da tela. A abordagem **Mobile First** consiste em escrever o CSS pensando primeiro no celular e depois adaptar para telas maiores.

```
/* Estilo base = mobile */
.card { flex-direction: column; }

/* A partir de 768px (tablet/desktop) */
```

```
@media (min-width: 768px) {  
  .card { flex-direction: row; }  
}
```

## 4. JavaScript ES6+

O ES6 (ECMAScript 2015) e versões seguintes trouxeram uma sintaxe mais moderna, concisa e poderosa para o JavaScript, muito usada no dia a dia com React.

### Arrow Functions

```
// Função tradicional
function somar(a, b) {
  return a + b;
}
```

```
// Arrow function equivalente
const somar = (a, b) => a + b;
```

Arrow functions têm sintaxe curta e não criam seu próprio **this** — herdam o contexto (this) de onde foram definidas, o que evita bugs comuns em componentes React.

### Desestruturação (Destructuring)

```
const usuario = { nome: "Ana", idade: 28 };
const { nome, idade } = usuario; // extrai propriedades do objeto

const cores = ["azul", "verde", "vermelho"];
const [primeira, segunda] = cores; // extrai itens do array
```

### Métodos de array mais usados

| Método    | O que faz                                                                    |
|-----------|------------------------------------------------------------------------------|
| map()     | Cria um novo array transformando cada item (ex.: renderizar listas no React) |
| filter()  | Cria um novo array só com os itens que passam numa condição                  |
| reduce()  | Reduz o array a um único valor (soma, agrupamento, etc.)                     |
| find()    | Retorna o primeiro item que satisfaz uma condição                            |
| forEach() | Executa uma ação para cada item, sem retornar novo array                     |

### Assincronismo: Promises e Async/Await

Uma **Promise** representa um valor que estará disponível no futuro (ex.: resposta de uma API). **async/await** é uma forma mais legível de trabalhar com Promises.

```
// Com Promises (.then)
fetch("https://api.exemplo.com/usuarios")
  .then(resposta => resposta.json())
  .then(dados => console.log(dados))
  .catch(erro => console.error(erro));

// Com async/await (mais legível)
async function buscarUsuarios() {
  try {
    const resposta = await fetch("https://api.exemplo.com/usuarios");
    const dados = await resposta.json();
    console.log(dados);
  } catch (erro) {
    console.error("Erro ao buscar dados:", erro);
  }
}
```

```
}
```

■ **Dica prática:** Use sempre **try/catch** junto com **async/await** para tratar erros de requisições HTTP (ex.: API fora do ar, erro de rede).

## 5. Manipulação do DOM

O DOM (Document Object Model) é a representação da página HTML em forma de árvore de objetos, que o JavaScript pode ler e modificar dinamicamente.

```
// Selecionar elementos
const titulo = document.querySelector("h1");
const itens = document.querySelectorAll(".item");

// Alterar conteúdo e estilo
titulo.textContent = "Novo título";
titulo.style.color = "blue";

// Criar e adicionar elementos
const novoItem = document.createElement("li");
novoItem.textContent = "Item novo";
document.querySelector("ul").appendChild(novoItem);

// Escutar eventos
document.querySelector("button").addEventListener("click", () => {
  console.log("Botão clicado!");
});
```

No React, raramente manipulamos o DOM diretamente: o próprio framework atualiza a interface automaticamente quando o **estado** muda (ver seção 6).

## 6. React — Componentes, Hooks e Estado

React é uma biblioteca para construir interfaces através de **componentes** reutilizáveis. A interface é atualizada automaticamente sempre que o **estado** (dados internos do componente) muda.

### Componentes

Um componente é uma função JavaScript que retorna JSX (HTML dentro do JavaScript).

```
function CartaoUsuario({ nome, cargo }) {
  return (
    <div className="cartao">
      <h3>{nome}</h3>
      <p>{cargo}</p>
    </div>
  );
}

// Uso do componente, passando "props"
<CartaoUsuario nome="Ana" cargo="Desenvolvedora" />
```

**Props** são os dados que um componente pai passa para um componente filho — o fluxo de dados no React é **unidirecional** (de cima para baixo).

### Hook useState — gerenciando estado

```
import { useState } from "react";

function Contador() {
  const [contagem, setContagem] = useState(0);

  return (
    <div>
      <p>Total: {contagem}</p>
      <button onClick={() => setContagem(contagem + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```

Sempre que **setContagem** é chamado, o React re-renderiza o componente automaticamente com o novo valor.

### Hook useEffect — efeitos colaterais (ex.: chamadas de API)

```
import { useState, useEffect } from "react";

function ListaUsuarios() {
  const [usuarios, setUsuarios] = useState([]);

  useEffect(() => {
    async function carregar() {
      const resposta = await fetch("https://api.exemplo.com/usuarios");
      const dados = await resposta.json();
      setUsuarios(dados);
    }
    carregar();
  }, []); // array vazio = executa só uma vez, ao montar o componente

  return (
```

```
    <ul>
      {usuarios.map(u => <li key={u.id}>{u.nome}</li>)}
    </ul>
  );
}
```

■ **Dica prática:** O array de dependências do `useEffect` (`[]`) controla quando o efeito roda de novo. Vazio = só na primeira renderização; com variáveis = roda quando elas mudarem.

## Boas práticas de componentização

- Cada componente deve ter **uma responsabilidade clara** (evitar componentes gigantes);
- Sempre use a prop **key** (única) ao renderizar listas com `map()`;
- Nunca modifique o estado diretamente — use sempre a função `set` retornada pelo `useState`;
- Prefira componentes de **apresentação** (visual) separados dos componentes de **lógica** (dados).

## 7. Acessibilidade (a11y) e Performance

### Responsividade

- Use unidades relativas (**%**, **rem**, **vw/vh**) em vez de valores fixos em pixels sempre que possível;
- Teste em múltiplos tamanhos de tela (celular, tablet, desktop);
- Combine Flexbox/Grid com Media Queries para adaptar o layout.

### Acessibilidade (a11y)

- Use tags semânticas (ver seção 2) para que leitores de tela entendam a estrutura da página;
- Todo campo de formulário precisa de um **<label>** associado;
- Garanta **contraste** adequado entre texto e fundo;
- Elementos interativos devem ser acessíveis via **teclado** (tab, enter);
- Use atributos **aria-\*** quando não houver tag semântica equivalente (ex.: aria-label).

### Performance web

- Otimize e comprima imagens antes de usá-las no projeto;
- Carregue componentes/rotas sob demanda (*lazy loading*) quando possível;
- Evite renderizações desnecessárias no React (ex.: uso de **useMemo** e **useCallback** quando fizer sentido);
- Minimize requisições HTTP e reaproveite dados já carregados em vez de buscar tudo de novo.

## 8. Checklist Final e Próximos Passos

### Checklist de conceitos estudados

- ■ HTML semântico — tags corretas para SEO e acessibilidade;
- ■ CSS moderno — Flexbox, Grid, variáveis e media queries;
- ■ JavaScript ES6+ — arrow functions, desestruturação, métodos de array;
- ■ Assincronismo — Promises, async/await e Fetch/Axios;
- ■ Manipulação do DOM — seleção, criação e eventos;
- ■ React — componentes, props, useState e useEffect;
- ■ Boas práticas — responsividade, acessibilidade e performance.

### Próximos passos sugeridos

- Aplicar os conceitos construindo componentes reais do projeto, consumindo uma API REST;
- Revisar o código em conjunto com a equipe, verificando boas práticas de limpeza e usabilidade;
- Formatar e revisar o material final de estudo antes de anexá-lo à documentação do projeto.

---

Resumo elaborado com base no plano de estudo de Frontend do time, cobrindo HTML, CSS, JavaScript e React de forma prática e objetiva para consulta durante o desenvolvimento.